LogiBridge Final Report

FreightBridge Logistics Pvt. Ltd. | LogiEdge Edge AI pilot

Architecture, design justification, pipeline mapping, OTA strategy and model optimisation

Contents

Section 1	Task A1: Constraint analysis	page 2
Section 2	Task A2: System architecture diagram	page 6
Section 3	Task D3: The 10-stage pipeline mapping	page 9
Section 4	Task C3: Data fusion justification	page 12
Section 5	Task E3: OTA strategy selection	page 16
Section 6	Task F1: Three model variants	page 20
Section 7	Task F2: Five-metric benchmarking	page 23
Section 8	Task F3: Deployment recommendation	page 26

Each section is a self-contained report with its own calculations, assumptions and open items.

Task A1 — Constraint Analysis

LogiEdge: Edge AI cold-chain monitoring | FreightBridge Logistics Pvt. Ltd. | Component A, Module 1

At a glance

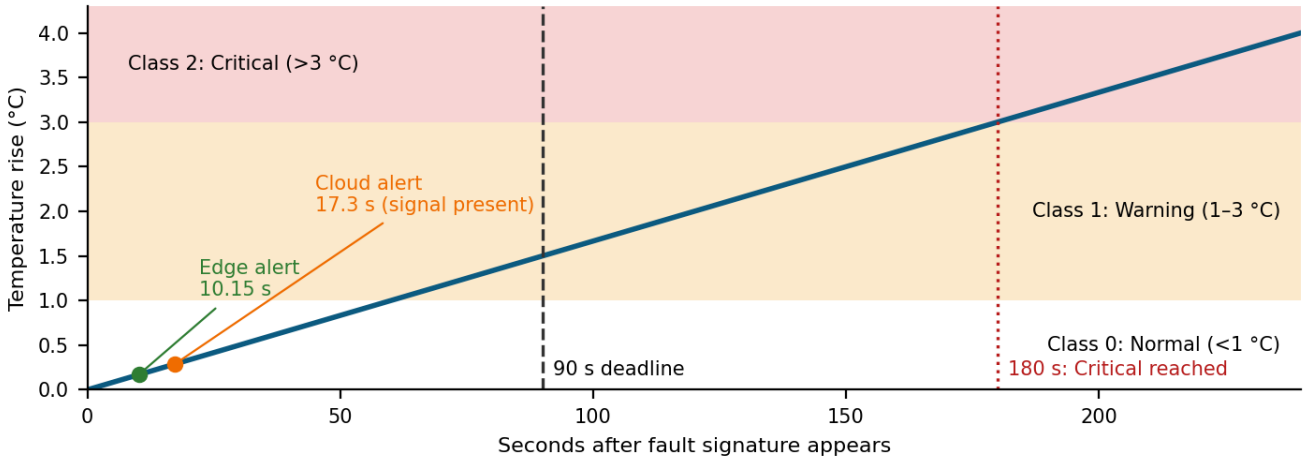
Constraint	Cloud-only system	LogiEdge (edge)	Verdict
Latency (limit 90 s)	17.3 s, only with signal	10.15 s, always	Edge
Data per truck per day	259.89 MB (₹25.99)	0.034 MB (₹0.0034)	Edge, 7,660× less
Connectivity gaps	Blind for 35–90 min	Works offline, syncs later	Edge
Privacy	Raw data leaves truck	Raw data stays on truck	Edge

1 Latency

Given: *a refrigeration fault raises cargo temperature by 1 °C per minute; alert needed within 90 s. Round-trip times assumed 0.2–1 s on rural 4G.*

Step	Working	Result
Rise at deadline	90 s × (1 °C ÷ 60 s)	1.5 °C (Warning band)
Time to Critical (>3 °C)	3 °C ÷ 1 °C/min	180 s
Cloud path	10 + 4.8 + 1 + 0.5 + 1 s	17.3 s (19% of budget)
Edge path	10 + 0.05 + 0.1 s	10.15 s (11% of budget)

Finding: cloud inference is feasible on latency alone, but only while a signal exists. Latency does not rule out the cloud; connectivity does.

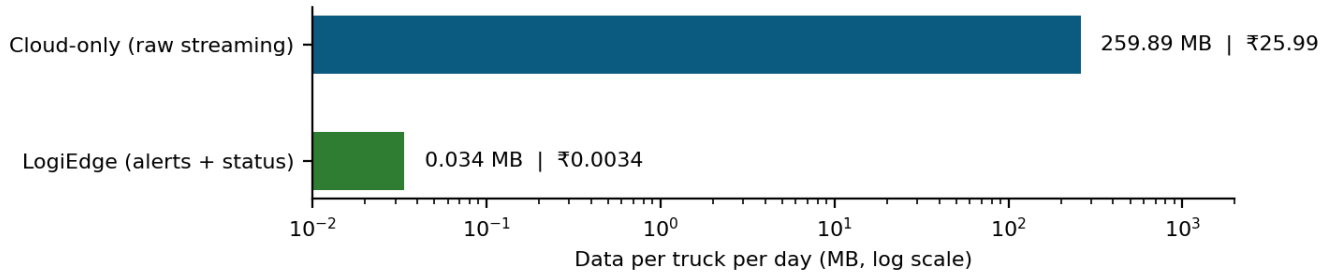


2 Bandwidth and cost

Given: *temperature 1 Hz; vibration 500 Hz × 3 axes; door events discrete; tariff ₹0.10/MB.*

Item	Working	Result
Vibration	500 × 3 × 2 B × 86,400 s	259.2 MB/day
Temperature	8 B × 86,400 s	0.69 MB/day
Door events	50 × 16 B	0.0008 MB/day
Raw total per truck	Sum of the above	259.89 MB/day
Cost per truck	259.89 × ₹0.10	₹25.99/day
Fleet of 85 trucks	259.89 × 85	22.09 GB/day; ₹2,209/day; ≈ ₹8.06 lakh/year
Edge alerts + status	288 × 100 B + 20 × 256 B	0.034 MB/day; ₹0.0034/truck/day; ≈ ₹105/year fleet

Finding: edge processing cuts volume and cost by a factor of 7,660 (99.99%).



3 Connectivity

Given: *seven gaps of 35–90 minutes each, so 245–630 minutes (4.1–10.5 hours) without signal. One gap is 23–60 times the 90 s budget.*

During a gap	Cloud-only	LogiEdge
Inference	None	Continues on the truck
Alerts	None; fault goes unnoticed	In-cab alarm within about 10 s
Data	Not recorded	Encrypted, hash-chained queue plus 1 Hz temperature log; 90 min of raw data is only 16.2 MB
Signal returns	Nothing to recover	Ordered upload with retries and acknowledgements

Finding: a fault at gap entry passes 3 °C within three minutes with nobody informed, which is the Nashik–Aurangabad vaccine-spoilage scenario. The edge design leaves no unmonitored period and keeps chain-of-custody records unbroken.

4 Privacy

- **Data stays local:** 259 MB/day of raw data remains in the cab; only class labels and signed alerts (about 34 KB/day) leave.
- **Encryption:** AES-256 at rest with hardware-backed keys; TLS 1.3 with mutual authentication in transit.
- **Tamper evidence:** device-signed, hash-chained logs make alteration detectable.
- **Contract:** a data-minimisation commitment that raw cargo-condition data is processed and retained only on a key-protected device, with disclosure limited to defined alert fields.
- **Caveat:** device hardening and access policies are still required.

Appendix A1 — Assumptions and Workings

Decimal units: 1 MB = 1,000,000 bytes; 1 lakh = 100,000. “Assumed” values are engineering estimates, not FreightBridge measurements.

A. Assumptions

Item	Value	Basis
Monitoring period	86,400 s/day (24 h)	Assumed; refrigeration runs while parked
Temperature record	8 B (4 B float32 + 4 B timestamp)	Assumed
Vibration sample	2 B (16-bit) per axis; timestamp implicit	Assumed
Door events	50 per day × 16 B	Assumed
Edge status packet	100 B every 5 min (288 per day)	Assumed
Edge alert message	256 B signed JSON; 20 per day	Assumed, conservative
Rural 4G round trip	1 s (range 0.2–1 s; 2G fallback up to about 3 s)	Assumed typical
Effective uplink	50 kbps = 6,250 B/s	Assumed
Feature window	10 s	Design choice
Edge inference / local alarm	0.05 s / 0.1 s	Assumed
Cloud inference / alert push	0.5 s / 1 s	Assumed
Tariff	₹0.10 per MB	Given

B. Latency workings

Step	Calculation	Result
Rise per second	1 °C ÷ 60 s	0.0167 °C/s
Uplink payload	10 s × 3,000 B/s	30,000 B
Upload time	30,000 B ÷ 6,250 B/s	4.8 s
Cloud total	10 + 4.8 + 1 + 0.5 + 1	17.3 s; 17.3 ÷ 90 = 19.2%
Edge total	10 + 0.05 + 0.1	10.15 s; 10.15 ÷ 90 = 11.3%

C. Bandwidth workings

Step	Calculation	Result
Vibration rate	500 samples/s × 3 axes × 2 B	3,000 B/s
Vibration per day	3,000 × 86,400	259,200,000 B = 259.2 MB
Temperature per day	8 × 86,400	691,200 B = 0.6912 MB
Raw total per truck	259.2 + 0.6912 + 0.0008	259.892 MB
Cost per truck	259.892 × 0.10; × 365	₹25.99/day; ₹9,486/year
Fleet volume	259.892 × 85	22,090.8 MB = 22.09 GB/day
Fleet cost	22,090.8 × 0.10; × 365	₹2,209.08/day; ₹806,315/year
Edge status	288 × 100 B	28,800 B
Edge alerts	20 × 256 B	5,120 B
Edge total	28,800 + 5,120	33,920 B = 0.03392 MB/day
Edge cost per truck	0.03392 × 0.10	₹0.0034/day

Step	Calculation	Result
Edge fleet cost	0.03392 × 85 × 0.10; × 365	₹0.288/day; ₹105.24/year
Reduction	259.892 ÷ 0.03392; 1 − (0.03392 ÷ 259.892)	7,662 times; 99.987%

D. Connectivity workings

Step	Calculation	Result
Total dark time	7 × 35; 7 × 90	245 min (4.08 h); 630 min (10.5 h)
Gap vs. budget	2,100 s ÷ 90; 5,400 s ÷ 90	23.3 times; 60 times
Raw buffer, 90 min	3,000 B/s × 5,400 s	16,200,000 B = 16.2 MB

E. Items to confirm

- Bytes per sensor sample and timestamp format; vibration sensor resolution.
- Daily door-event count and expected alert count.
- Measured round-trip times and uplink speeds on the Nashik–Aurangabad route.
- Whether 35–90 minutes applies to each of the seven locations (as calculated) or to the route in total, since 4.1–10.5 hours is long for that journey.
- Whether monitoring runs 24 hours a day.

Task A2 — System Architecture Diagram

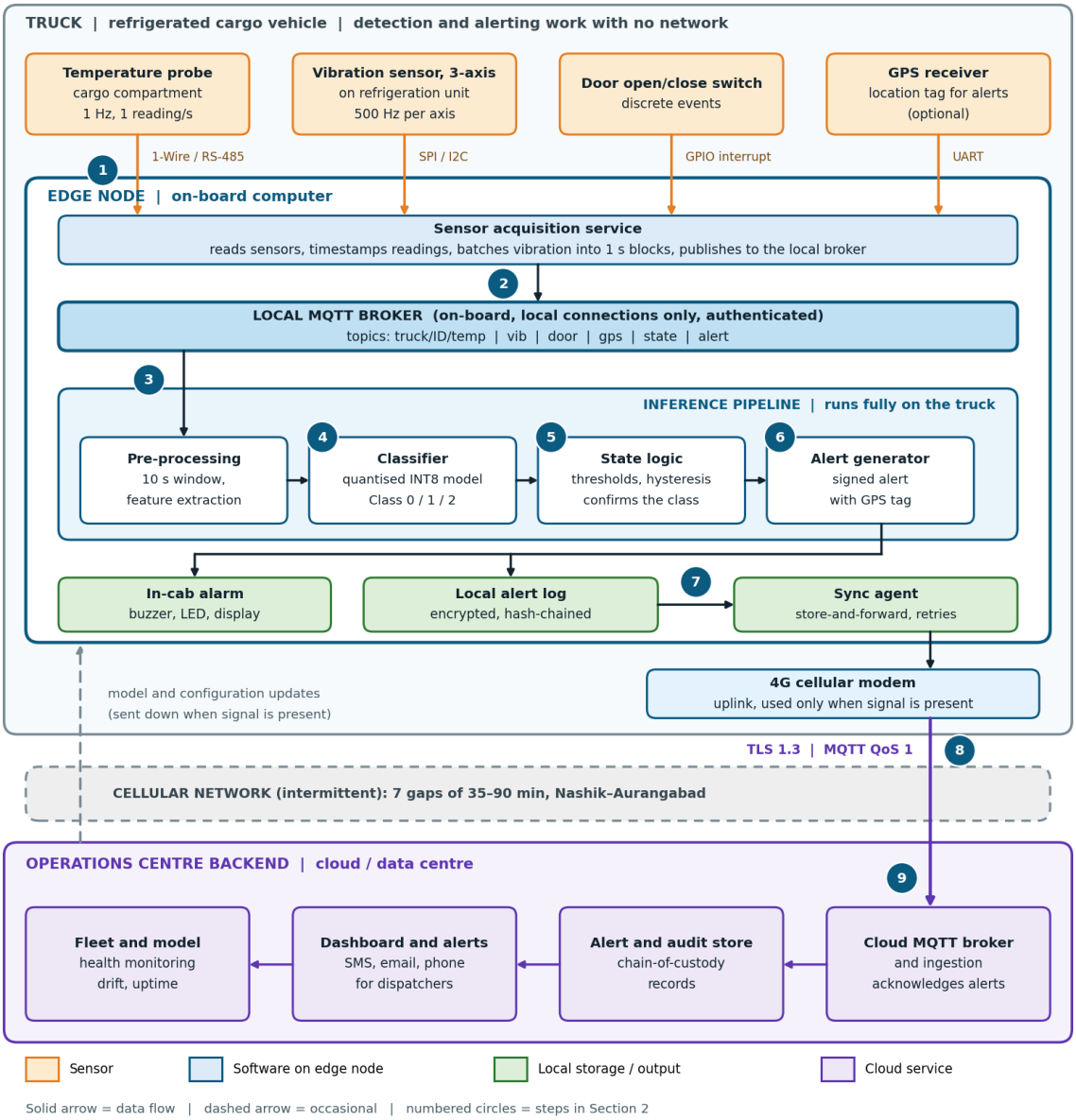
LogiEdge: Edge AI cold-chain monitoring | FreightBridge Logistics Pvt. Ltd. | Component A, Module 1

At a glance

Key fact	Design
Where detection runs	On the truck's edge node; no cloud dependency
Steps that need a network	Only steps 7–9 (uplink and operations centre)
Detection-to-alarm time	≈ 10.15 s (workings in Task A1)
Uplink volume	≈ 0.034 MB per truck per day (Task A1)

1 Architecture diagram

Given: *show all seven required components, from sensors to operations centre.*



Arrows show the logical data flow. Internally, the alert generator publishes each alert on the alert topic, and the log writer and alarm driver subscribe to it.

2 Data flow, step by step

Step	From → To	What moves	Needs network?
1	Sensors → acquisition service	Temperature 1 Hz; vibration 500 Hz × 3 axes; door events; GPS position	No
2	Acquisition service → local broker	MQTT messages: 8 B temperature readings, 1 s vibration blocks of 3,000 B, door events	No
3	Local broker → pre-processing	Subscribes to sensor topics; builds non-overlapping 10 s windows and features	No
4	Pre-processing → classifier	Features in; quantised INT8 model returns Class 0, 1 or 2 with a confidence	No
5	Classifier → state logic	Thresholds (Warning 1–3 °C, Critical >3 °C) and hysteresis confirm the class	No
6	State logic → alert generator → alarm and log	Signed alert (≈ 256 B) written to the local alert log; in-cab alarm triggered	No
7	Local alert log → sync agent	Sync agent reads entries not yet acknowledged	No
8	Sync agent → modem → cellular → cloud	TLS 1.3, MQTT QoS 1; retries until acknowledged	Yes
9	Cloud broker → alert store → dashboard	Ingest, store as chain-of-custody record, notify dispatchers	Yes (cloud side)

Finding: steps 1–6 stay on the truck, so detection and alarms continue through every signal gap. Only steps 7–9 wait for coverage. Even a 3-window confirmation rule (windows complete at 10, 20 and 30 s) would alert in about 30 s, inside the 90 s limit.

3 Behaviour with and without signal

Aspect	Signal present	Signal lost (35–90 min)
Detection and classification	On the truck	Unchanged, on the truck
Driver alert	In-cab alarm	In-cab alarm
Operations centre alert	Sent as soon as the alert is logged	Queued in the local alert log
Records	Logged locally and uploaded	Logged locally; nothing is lost
When signal returns	Not applicable	Queue uploads in order; unacknowledged entries are retried

Finding: the alarm and log form the primary path, and the uplink is a reporting path. A cellular failure therefore cannot stop detection or leave a gap in the record.

4 Component responsibilities

Component	Responsibility	Design choice
Edge node	Runs acquisition, broker, pipeline, log and sync	One on-board computer; hardware justified in a later task
Sensor connections	Feed raw readings to the acquisition service	One dedicated link per sensor (examples: 1-Wire or RS-485, SPI or I2C, GPIO, UART); finalised at hardware selection
Local MQTT broker	Routes every on-truck message	Local connections only, authenticated; sensors and pipeline can change independently
Inference pipeline	Turns readings into Class 0, 1 or 2	Fully on device; quantised INT8 model
Local alert log	Permanent record of alerts and temperature	Append-only, AES-256 encrypted, hash-chained; also holds the 1 Hz temperature log (0.69 MB/day)
Cellular uplink	Moves alerts to the operations centre	Sync agent plus 4G modem; TLS 1.3 with mutual authentication; MQTT QoS 1
Operations centre backend	Receives, stores, notifies, monitors	Cloud MQTT broker, alert and audit store, dispatcher dashboard, fleet and model health monitoring

5 Local MQTT topic design

Topic	Rate	Payload	QoS	Subscribers
truck/ID/temp	1 per second	8 B reading	0	Pipeline, log
truck/ID/vib	1 per second	3,000 B block (500 Hz × 3 axes × 2 B)	0	Pipeline
truck/ID/door	On each event	16 B event	1	Pipeline, log
truck/ID/gps	Periodic	Position	0	Alert generator (location tag)
truck/ID/state	Every 10 s window	Class and confidence	0, retained	In-cab display, status summary
truck/ID/alert	On each alert	≈ 256 B signed message	1	Log writer, alarm driver

Finding: local MQTT traffic never leaves the truck. Only status packets and alerts (about 34 KB per day) cross the cellular link, matching the Task A1 bandwidth figures.

6 Items to confirm

- Compute hardware for the edge node (selected and justified in a later task).
- Sensor models and interfaces; those shown are examples.
- MQTT broker software, and whether the operations centre backend is cloud-hosted or on-premises.
- Hysteresis rule (how many windows confirm a class change), tuned during validation.
- Log retention period, and whether over-the-air model updates are wanted (shown dashed).
- Whether a buzzer, LED or display is already fitted in the cabs.

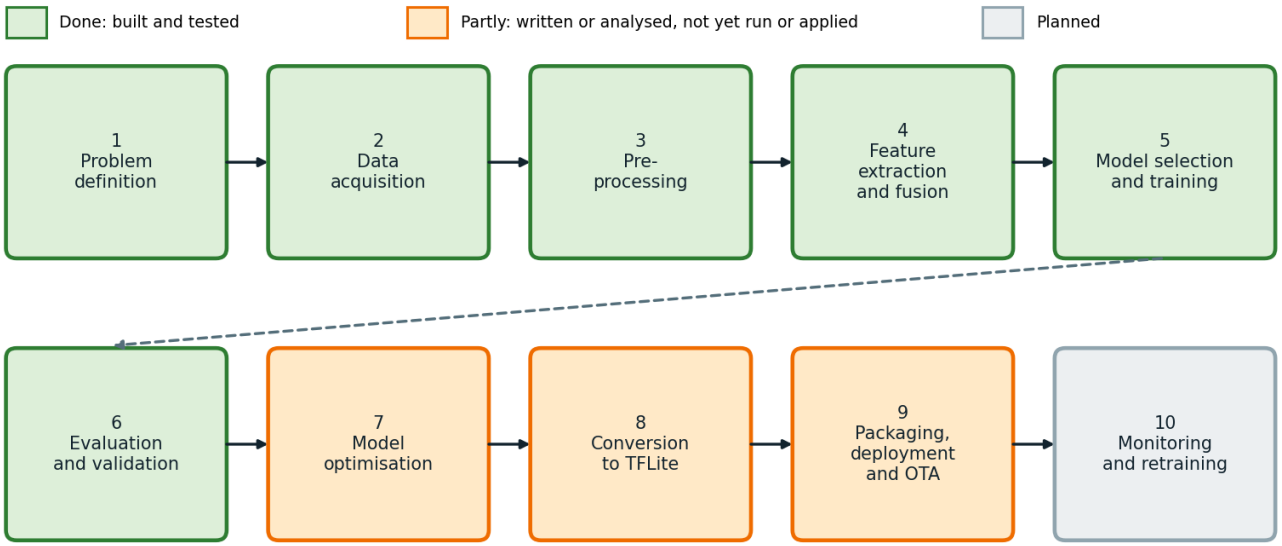
Task D3 — The 10-Stage Pipeline Mapping

Final Report Section 3 | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

At a glance

Item	Summary
Mapping	LogiEdge mapped to all 10 stages of the Edge ML pipeline, each described for this system in one or two sentences
Status	Stages 1–6 are done and tested; stages 7–9 are partly done (analysed or written, not yet run or applied); stage 10 is planned
Stage names	The 10 stage names below are the standard edge ML sequence. I do not have the Module 4 list, so please check they match (Section 3)

1 The pipeline at a glance



2 Stage-by-stage mapping

#	Stage	What happens in LogiEdge	Evidence	Status
1	Problem definition and requirements	FreightBridge needs on-truck classification of cargo state as Normal, Warning or Critical, with an alert inside 90 s and no dependence on cellular coverage, within a 10 W AI budget for 85 trucks now and 265 later. The cloud was ruled out because seven 35–90 minute coverage gaps would leave it blind and raw telemetry would cost about ₹25.99 per truck per day.	A1, A2, B1	Done
2	Data acquisition	A Python simulator publishes cargo temperature (1 Hz), compressor vibration RMS (0.5 Hz) and door events to a local Mosquitto broker in four modes: none, temp_drift, vibration and combined. All data so far is simulated; no real truck sensor is connected yet.	C1: sensor_simulator.py	Done (simulated data)
3	Data preprocessing	Each stream gets a 5-sample moving average, is cut into 30 s windows every 10 s, and is z-scored with statistics from 10 minutes of clean Normal data saved in training_stats.npy, which is only ever loaded and never recomputed. A window with under 80% of its samples is skipped rather than filled.	C2: preprocessing.py	Done

#	Stage	What happens in LogiEdge	Evidence	Status
4	Feature extraction and fusion	Each sensor is summarised on its own (temperature mean, standard deviation and °C/min rate; vibration RMS, peak and kurtosis), then the two blocks are concatenated into one 6-value vector. Feature-level fusion was chosen over data-level and decision-level because it matches their accuracy at far lower compute and needs no hand-written cross-sensor rule.	C2, C3	Done
5	Model selection and training	A 6→32→16→3 ReLU MLP with 803 parameters was trained on 291 windows from 20 minutes of Normal, 15 of temperature drift and 15 of combined-fault simulator data. It targets the Raspberry Pi 5 with AI HAT+ (7.5 W against the 10 W budget, ₹15,000 per truck).	D1, B1	Done
6	Evaluation and validation	The 88% gate was passed with 98.25% on the required 20% split, and 50 random splits, 5-fold, blocked and independent-run checks stayed at or above 94.7%. They also found that faults starting mid-run are missed 3.9% of the time, which the onset-trained model removes, and Task C2 measured that 3σ-shifted statistics cost 1.3 to 8.4 points.	D1, C2	Done
7	Model optimisation	Task B2's roofline analysis of a 45 MFLOP model showed it is compute-bound on the Pi 5 CPU (2.81 ms) and favours INT8 quantisation and pruning. The trained model has only 803 parameters and already runs far inside the 90 s limit, so no optimisation has been applied, and quantisation must be validated first because normalised inputs reach about 1,300.	B2	Partly (analysed, not applied)
8	Model conversion	convert_to_tflite.py rebuilds the trained weights as a float32 TFLite model and refuses to write it unless Keras and TFLite match the numpy model on every dataset window. It has not been run yet, because TensorFlow was not available to me.	D2: convert_to_tflite.py	Partly (written, not run)
9	Packaging, deployment and OTA	Preprocessing, inference and the MQTT publisher run in a python:3.11-slim container that publishes to logibridge/trucks/{truck_id}/inference, selects its model with MODEL_PATH and copies the model as the last layer. An OTA model update then resends only that layer, estimated at 16 KB instead of 65 MB per truck, a 99.98% saving for 85 trucks; the image itself has not been built yet.	D2: Dockerfile, ota_demo.sh	Partly (written, not run)
10	Monitoring, drift and retraining	Planned: every result already carries class, confidence, the six features and the model hash, so the operations centre can watch for feature drift against training_stats.npy and falling confidence, which matters because Task C2 showed a 3σ shift costs up to 8.4 points. Retraining on real truck data would be shipped through the same model-layer OTA path.	Problem statement (MLOps); D2 payload	Planned

3 Notes and items to confirm

- **Stage list.** I used the standard 10-stage edge ML sequence: problem definition, data acquisition, preprocessing, feature extraction, model selection and training, evaluation, optimisation, conversion, packaging and deployment, then monitoring and retraining. If Module 4 names or orders them differently, send me the list and I will relabel; the content of each row carries over.
- **Status is stated as it is today.** Stages 7–9 become “Done” once you run convert_to_tflite.py, build the image with ota_demo.sh and (if wanted) apply INT8 quantisation; stage 10 needs the monitoring component. I will update the table with your results.
- **Simulated data.** Stages 2, 5 and 6 rest on simulator data, so accuracy figures are not yet evidence of real-truck performance.
- **Figures quoted** (98.25%, 3.9%, 99.98%) come from Tasks D1 and D2; the 16 KB and 65 MB sizes are my estimates until measured.

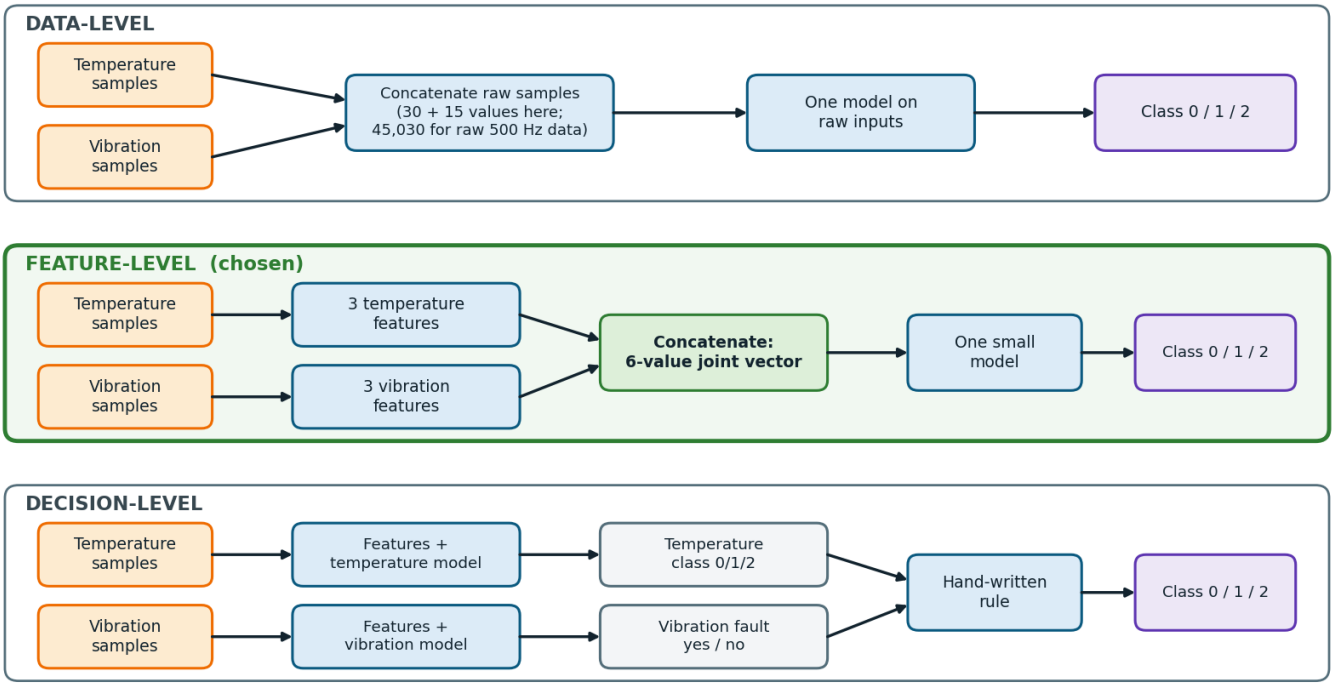
Task C3 — Data Fusion Justification

Final Report section | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component C, Module 3

At a glance

Question	Answer
Fusion level chosen	**Feature-level.** Each sensor is summarised on its own into 3 features; the two blocks are concatenated into one 6-value joint vector for one small model.
Against data-level	Same accuracy (99.7% against 99.6%) at about one fifth of the compute (144 against 768 multiply-accumulates). On raw 500 Hz vibration the gap grows to about 5,003 times.
Against decision-level	A simple max rule loses 2.0 points (97.7%); matching accuracy (99.7%) needs a hand-written cross-sensor rule that feature-level learns from data.
Honest limit	Accuracy differences between sound designs are within 0.25 points in simulation. The case rests on cost, simplicity and fit, not on a large accuracy gain.

1 The three fusion levels



Level	Where the sensors are combined	Typical use
Data-level	Raw samples are joined before any feature or model	Same kind of sensor, same rate, well aligned
Feature-level	Each sensor is reduced to features; the features are joined	Different sensors with different rates and units
Decision-level	Each sensor reaches its own decision; the decisions are combined	Independent sensors, or when modules must stay separate

2 What was implemented

```
t = temperature_features(ts, temp_window)   # [mean, std, rate in degC/min]   (temperature only)
v = vibration_features(vib_window)          # [RMS, peak, kurtosis]            (vibration only)
joint = fuse_features(t, v)                 # one 6-value vector: t first, then v
z = normaliser.transform(joint)             # saved statistics from training_stats.npy
```

- Features are extracted independently: a test changes only the vibration stream and confirms the three temperature features do not move, and the reverse.
- Concatenation happens before the model, so the model sees one 6-value input and can learn how the two sensors interact.

3 Justification for this application

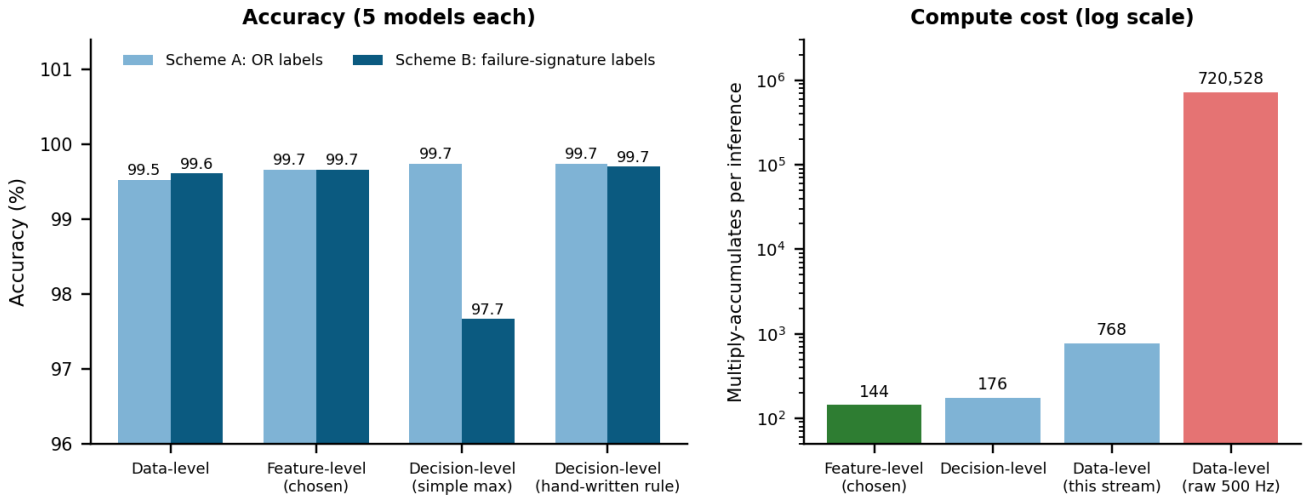
Criterion	Data-level	Feature-level (chosen)	Decision-level
Different sensors and rates	Needs alignment of 1 Hz °C readings, 0.5 Hz g values and, in the real truck, 500 Hz × 3 axes, plus unit scaling	Each stream is summarised over the same 30 s window every 10 s, so the rates never meet	No alignment problem
Edge compute (10 W budget, Tasks B1 and B2)	768 multiply-accumulates here; about 720,528 on raw 500 Hz data (45,030 inputs)	144 multiply-accumulates; 6 inputs	176 multiply-accumulates across two models
Cross-sensor meaning (a failing compressor with rising temperature is a unit-failure signature)	Can learn it, from many more inputs	Learns it directly from the joint vector	Needs a hand-written rule; the simple max rule loses 2.0 points
Audit and explanation (pharma chain of custody)	Opaque inputs	Each input is physical: mean °C, rate °C/min, RMS g. Vectors can be logged with alerts	Per-sensor decisions are easy to explain
Fit with Task C2 statistics	45 or 45,030 statistics to store and keep consistent	Six, already in training_stats.npy	Six, split across two models
Sensor failure	One input group disappears	Window is skipped and counted; needs a watchdog (Section 5)	Easy to fall back to the other sensor in principle
Information kept	Everything, including waveform shape	Summary only; add features if needed	Summary only, then reduced to a class

4 Evidence from the simulator

Same data, labels and training budget for every design (16 hidden units, 5 models each). **Scheme A** labels a window by the worse of the two sensors (an "OR"). **Scheme B** also labels a vibration fault together with temperature more than 1 °C off setpoint as Class 2, a unit-failure signature. Test set: 3,240 windows.

Design	Acc. A	Acc. B	Params	MACs	No vib. (B)	No temp. (B)
Data-level (30 + 15 samples)	99.53%	99.62%	787	768	77.0%	64.2%
Feature-level (chosen)	99.67%	99.67%	163	144	77.2%	64.1%
Decision-level, simple max	99.75%	97.67%	213	176	77.2%	64.2%
Decision-level, hand-written rule	99.75%	99.71%	213	176	77.2%	64.2%

Acc. = accuracy on that labelling scheme; MACs = multiply-accumulates per inference; No vib. / No temp. = Scheme B accuracy when that sensor's inputs are replaced by Normal values.



The accuracy axis starts at 96% so small differences are visible. The raw 500 Hz bar is calculated (45,030 inputs × 16 hidden units + 16 × 3), not trained.

What the numbers show

- **Accuracy is essentially tied.** Data-level, feature-level and rule-based decision-level are all within 0.25 points on both schemes. I am not claiming feature-level is more accurate.
- **Compute separates them.** Feature-level needs 144 multiply-accumulates against 768 for data-level on this stream, and about 720,528 on raw 500 Hz vibration, roughly 5,003 times more.
- **Decision-level needs the cross-sensor logic written by hand.** With simple max it drops from 99.75% on Scheme A to 97.67% on Scheme B, because it cannot see that the two anomalies together mean failure. A correct rule recovers the accuracy but has to be rewritten and revalidated whenever the class definitions change; feature-level only needs retraining.
- **Missing-sensor tests found no difference.** When one sensor's inputs are replaced by Normal values, all four designs lose the same accuracy (within 0.3 points), since the missing sensor's anomalies are simply unseen. Decision-level's fault-tolerance advantage did not appear in this test.

5 Where feature-level is weaker, and how it is covered

Weakness	Cover
Waveform detail is discarded	The current vibration stream is already RMS. If bearing diagnosis later needs frequency content, add spectral features to the vector before concatenation; the approach extends without redesign.
A failed sensor can pass unnoticed	The pipeline skips and counts windows with under 80% of their samples. Add a watchdog that raises a sensor-fault alert when windows are skipped, rather than classifying on missing data.
Hand-picked features may miss a new failure mode	Keep fleet-level drift and accuracy monitoring (the MLOps component) and retrain when new faults appear.
Sensitive to normalisation statistics	Task C2 showed a 3σ shift costs 1.3 to 8.4 points, so the statistics are fixed, loaded from file and never recomputed from live data.

6 Conclusion

Feature-level fusion is the best fit for LogiEdge. The two sensors differ in rate, unit and meaning, the truck has a 10 W compute budget, and, under the failure-signature reading of Class 2, the class depends on how temperature and vibration combine. Summarising each sensor first keeps the model tiny and its inputs physical and loggable for pharmaceutical audit, while the joint vector lets one model learn the cross-sensor logic without hand-written rules. Data-level fusion would match the accuracy at far higher cost and decision-level fusion at the price of maintained rules; neither offers a measured accuracy benefit here.

7 Limits and items to confirm

- Results come from simulated data with a small reference classifier. Rerun `experiments_c2_c3.py` with your trained model and real truck data before treating the accuracy comparison as final.
- Scheme B is my interpretation of “refrigeration unit failure signature”. If your labels follow a pure OR rule (Scheme A), decision-level with a max rule is as accurate and the argument rests on compute and maintenance only.
- The data-level comparison uses 30 + 15 filtered samples. The raw 500 Hz cost is a calculation with 16 hidden units, not a trained model.
- All designs use 16 hidden units; other sizes would change the compute figures but not their ordering.

Task E3 — OTA Strategy Selection

Full replacement, canary and shadow mode for a 280 KB model | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component E, Module 5

At a glance

Item	Result
Recommendation	**Shadow mode on all 85 trucks**: the new model runs beside the old one for 7 days without acting, then is promoted by a small config message
Cost per update cycle	Full replacement ₹2.38 (23.8 MB); canary ₹2.43 (₹0.61 if rolled back); shadow ₹3.20 with on-device summaries, ₹53.80 if every window is streamed
Why cost does not decide it	The whole spread is under ₹52 per cycle. One vaccine-spoilage loss was ₹28 lakh, about 52,000 times the dearest option.
Why shadow	It exposes no truck to an unvalidated model (full: 85, canary: 10), compares the models on every window of every truck, and needs no connectivity for a rollback because the new model never acted

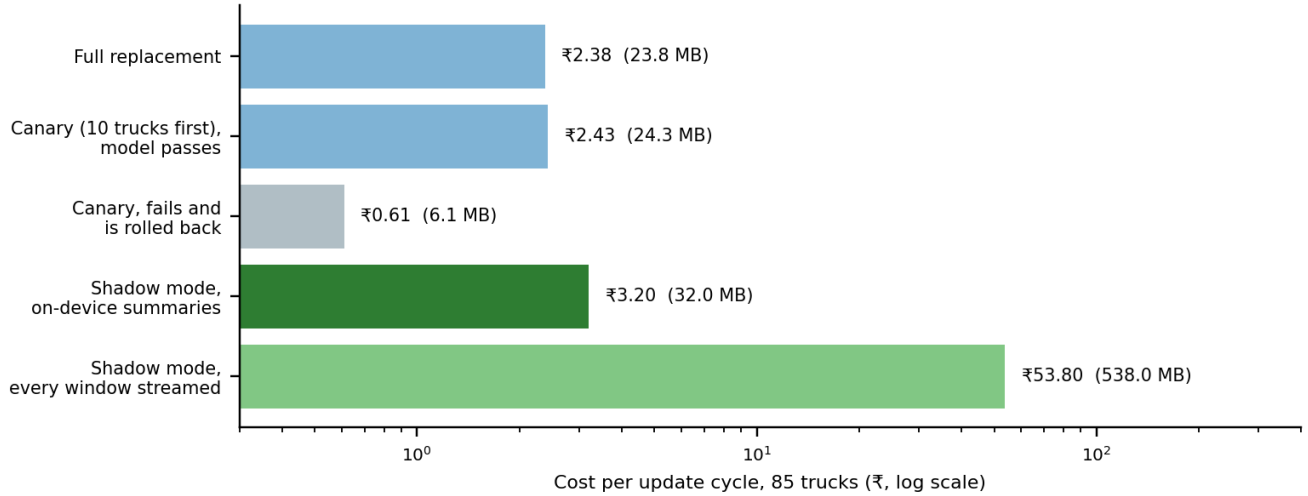
1 Inputs

From the brief	Value
Model file	280 KB (INT8 TFLite)
Update cadence	Every 6 weeks (8.67 cycles a year)
Fleet	85 trucks in the pilot, 265 at full scale
Tariff	₹0.10 per MB on an M2M SIM

My assumptions (not in the brief)	Value
Units	1 KB = 1,000 B and 1 MB = 1,000,000 B, as in Task A1
Canary and shadow observation period	7 days
Inference windows	One every 10 s, 8,640 a day (Task C2)
Hourly summary report	300 B per truck
Disagreement record	150 B each; 0.5% of windows disagree
Streaming every window	100 B per window
Promote message	1 KB per truck
Package manifest overhead	Ignored; about 10 KB per truck (Task D2) would add under 4%

2 Bandwidth cost per update cycle

Strategy	Model MB	Telemetry MB	Total MB	₹ per cycle	₹ per year	₹ per cycle, 265 trucks
Full replacement	23.80	0.00	23.80	2.38	20.63	7.42
Canary, model passes	23.80	0.50	24.30	2.43	21.06	7.47
Canary, fails and is rolled back	5.60	0.50	6.10	0.61	5.29	0.61
Shadow, on-device summaries	23.80	8.22	32.02	3.20	27.75	9.98
Shadow, every window streamed	23.80	514.16	537.97	53.80	466.24	167.72



For scale, one vaccine-spoilage loss was ₹28 lakh (₹28,00,000), about 52,000 times the dearest bar.

Workings

Strategy	Calculation
Full replacement	85 × 0.28 MB = 23.80 MB; × ₹0.10 = ₹2.38
Canary, model passes	(10 + 75) × 0.28 MB = 23.80 MB, plus 10 trucks × 7 days × 7.2 KB of hourly reports = 0.50 MB: 24.30 MB, ₹2.43
Canary, fails	10 trucks out and 10 back = 5.60 MB, plus 0.50 MB of reports: 6.10 MB, ₹0.61. The other 75 never receive it.
Shadow, summaries	23.80 MB, plus 85 × 7 days × 13.68 KB (7.2 KB summaries + 6.48 KB of disagreements) = 8.14 MB, plus 85 × 1 KB = 0.09 MB: 32.02 MB, ₹3.20
Shadow, streaming	23.80 MB, plus 85 × 7 days × 864 KB = 514.08 MB, plus 0.09 MB: 537.97 MB, ₹53.80

3 The cold-chain and rural facts that decide it

Fact	Consequence
All 85 pilot trucks carry cold-chain pharmaceuticals, and one refrigeration-failure loss (₹28 lakh of vaccine) happened in 18 months	There are no “safe” trucks to test on, and a missed fault is very costly
That is about one real fault per 78 weeks across the fleet. A 7-day canary on 10 trucks has a 0.15% chance of meeting one; shadow on 85 trucks, 1.3%	Real faults are too rare to confirm a model catches them. Whatever the strategy, validation needs a way to test faults without waiting for one, and without letting an untested model decide
Task D1: the model passed the 88% gate with 98.25%, yet missed 3.9% of anomalous windows when a fault started mid-run	A model that passes offline checks can still be unsafe in the field, so it should not act until it is compared on real data
Coverage gaps of 35–90 minutes at seven places on the Nashik–Aurangabad route	Reports and rollback commands reach a truck late, so a bad model keeps running unmonitored for up to 90 minutes per gap
280 KB takes 45 s at 50 kbps and under 4 minutes at 10 kbps	The transfer fits any coverage window. Connectivity affects monitoring and rollback far more than download

4 Recommendation: shadow mode

Send the 280 KB model to all 85 trucks and keep the old model resident. For 7 days both run on every window; only the old model raises alarms. Each truck compares the two on the truck, sends an hourly summary and a short record for each disagreement, and uploads them when coverage returns. The ops centre promotes the new model, by a small config message, only if these gates pass:

- **Safety disagreements:** zero windows where the new model says Normal and the old one says Warning or Critical, or every one is reviewed and explained.
- **Overall disagreement:** at most 1% of windows (my suggested threshold).
- **Fault replay:** recorded and simulated faults, including faults that start mid-run, are injected into the new model on the truck or in test and must all be detected.
- **Output stability:** the PSI monitor from Task E1 stays below 0.10 for the new model on every truck.
- **Coverage:** summaries received from at least 95% of trucks, since gaps delay reports.

The price is ₹0.82 more per cycle than full replacement (₹7.13 a year), plus a second inference per window: about 0.03% of the CPU on a Raspberry Pi 5 (2.8 ms per 10 s window, Task B2).

5 Why not the other two

Full replacement

- **Every truck is exposed at once.** A model with a blind spot like the one in Task D1 would blind all 85 pharmaceutical loads together, and a single missed refrigeration failure is a ₹28 lakh event.
- **Rollback depends on connectivity.** A truck inside a 90-minute gap keeps running the bad model, and trucks update at different times anyway, so the fleet is never uniform.
- **It saves almost nothing.** Shadow costs ₹0.82 more per cycle.

Canary (10 trucks first)

- **It lets 10 real pharmaceutical loads depend on an unvalidated model.** If the model misses a fault on one of them, no alarm is raised and the load can spoil.
- **A clean canary week proves little.** With about a 0.15% chance of any real fault on those trucks, “nothing went wrong” does not show that faults are caught.
- **The canary is unrepresentative and slow to report.** Easy-coverage routes hide the rural conditions where trouble is likeliest, and reports from the canary arrive late after each gap.
- **It is nearly as expensive as shadow** (₹2.43 against ₹3.20) while protecting far less.

Canary's real strength is that it tests the new model's alerts in real operations. If that matters, run a canary after shadow mode, not instead of it.

6 Decision matrix

Criterion	Full replacement	Canary	Shadow (recommended)
Trucks exposed to an unvalidated model's decisions	85 (100%)	10 (12%)	0
Real-data comparison with the old model	None	10 trucks, if at all	All 85 trucks, every window
Judged on	Field outcomes, which are rare	Field outcomes, which are rare	Agreement with the old model plus fault replay
Rollback needs connectivity	Yes, urgently	Yes, for 10 trucks	No: the new model never acted
Bandwidth per cycle	₹2.38	₹2.43	₹3.20
Time to benefit	Days	About 1 week, then the rest	About 1 week, then a config switch
On-truck complexity	Lowest	Low	Higher: two models and a comparison

7 Weaknesses of shadow mode and how they are covered

Weakness	Cover
The benefit arrives about a week later (7 of 42 days, 17% of the cycle)	Acceptable for a routine 6-week update. An urgent safety fix is a separate decision and needs sign-off.
Agreement with the old model is not correctness	Add the fault-replay gate, and review every safety disagreement by hand.
Telemetry can balloon if every window is streamed (₹53.80 per cycle)	Aggregate on the truck: ₹3.20 per cycle.
Coverage gaps delay the summaries	Store and forward as in Task A2; decide on the reports that have arrived and require 95% coverage.
More code on the truck	Keep the old model as a standby slot, so promoting or reverting is a config switch with no download.

8 Items to confirm

- **Assumptions.** The 7-day soak, 0.5% disagreement, 300 B hourly summary and 150 B disagreement record are mine. `ota_strategy_cost.py` recomputes everything if you change them.
- **Kilobyte definition.** I used 280 KB = 280,000 B. With 1 KiB = 1,024 B the model is 286,720 B and every figure rises by 2.4%.
- **Fleet scale.** The right-hand column shows the 265-truck cost; the ranking does not change.
- **Your stack.** Shadow mode needs the truck to hold two models and compare them. Tell me if your runtime cannot, and I will revisit canary with extra safeguards.

Task F1 — Three Model Variants

FP32 baseline, PTQ INT8, and structured pruning + PTQ INT8 | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component F, Module 6

At a glance

Item	Result
Variants	M1 FP32 baseline 3,212 B of numbers; M2 PTQ INT8 1,568 B; M3 35% pruned + PTQ INT8, network 6→21→10→3, 400 parameters (−50%)
Calibration	234 windows for the INT8 ranges (at least 200 required)
Status	**The TensorFlow / tfmot build has not been run** (neither library was available to me). All three variants were built and measured in numpy with TFLite's full-integer arithmetic; `build_variants_tf.py` makes the real .tflite files.
Baseline	M1 is the Task D1 recommended model (trained with mid-run-onset runs). The required D1 model reaches only 94.1% Critical recall on the 57-window split, one window short of the 95% target in Task F3.

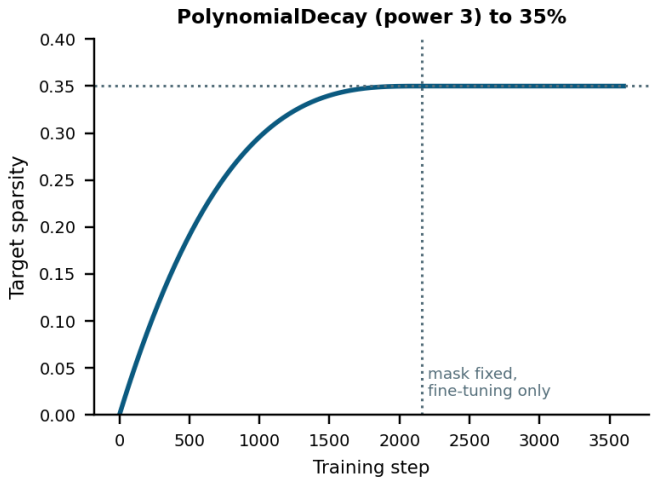
1 The three variants

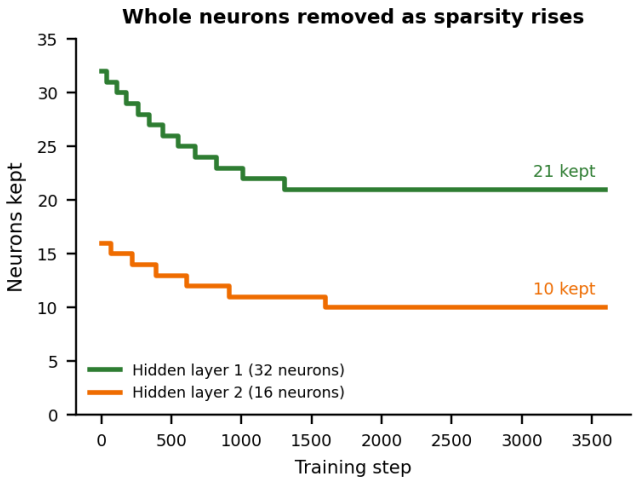
Variant	Method	Network	Parameters	Stored numbers	Tools
M1	Standard training (Task D1), FP32	6 → 32 → 16 → 3	803	3,212 B (FP32)	tf.keras (numpy here)
M2	Post-training quantisation, full INT8, calibrated on 234 windows	6 → 32 → 16 → 3	803	1,568 B (INT8 weights, INT32 biases, scales)	TFLiteConverter, DEFAULT + representative dataset
M3	35% structured pruning (PolynomialDecay), fine-tune, remove neurons, then full INT8 PTQ	6 → 21 → 10 → 3	400	910 B	tensorflow_model_optimization + TFLiteConverter

2 M2: post-training INT8 quantisation

- **Scheme (as TFLite's full-integer mode):** int8 activations with a scale and zero point from the min and max seen on the calibration windows; symmetric int8 weights per output channel; int32 biases; int32 accumulation; requantisation to int8 after every layer.
- **Calibration:** the 234 training windows (the code refuses fewer than 200).
- **One thing to know:** the normalised inputs run from about −3 (Normal) to about 1,300 (a 70 °C window), so the input scale is 4.99, about five Normal-class σ per step. The whole Normal range lands on one or two integer levels. Accuracy still holds on clean data (Task F2), but it makes INT8 less tolerant of noise.
- **Per channel or per tensor:** both gave identical predictions here. Newer TensorFlow versions quantise Dense layers per channel by default; whichever your version uses, the results should match.

3 M3: structured pruning with a polynomial-decay schedule





Step	What happens
Granularity	Whole hidden neurons (the “filters” of a dense layer). In tfmot this is `block_size=(inputs, 1)`: one whole kernel column is pruned at a time. Unstructured pruning would zero single weights and leave the file the same size.
Importance	Mean absolute value of a neuron's incoming weights (tfmot's AVG block pooling)
Schedule	PolynomialDecay from 0% to 35%, power 3, reached at 60% of training; mask updated every 10 steps; output layer not pruned
Fine-tuning	200 epochs of Adam on the training windows plus the mid-run-onset runs; pruned weights stay zero
Removal	The zeroed neurons are deleted with their matching rows in the next layer: 32 → 21 and 16 → 10 neurons. A neuron's constant output (relu of its bias) is folded into the next bias, so the smaller network computes the same function (tested).
Result	803 → 400 parameters (−50%); 752 → 366 multiply-accumulates per inference (−51%)

4 The TensorFlow build (for you to run)

```
python3.11 -m venv .venv && . .venv/bin/activate
pip install tensorflow==2.14.1 tensorflow-model-optimization==0.8.0 numpy==1.26.4 psutil
python3 build_variants_tf.py          # M1 SavedModel, m1_fp32.tflite, m2_ptq_int8.tflite, m3_pruned_int8.tflite
```

Specified setting	Where it is in build_variants_tf.py
tf.lite.Optimize.DEFAULT	`converter.optimizations = [tf.lite.Optimize.DEFAULT]`
Representative dataset, 200 or more samples	234 training windows; raises an error below 200
Full INT8	`TFLITE_BUILTINS_INT8`, int8 input and output
35% structured pruning, PolynomialDecay	`PolynomialDecay(final_sparsity=0.35)` with `prune_low_magnitude(block_size=(inputs, 1))`
Checks built in	Keras M1 matches the trained weights; the physically smaller M3 matches the stripped model; each .tflite is scored on the validation set; exits with an error if Critical recall is 95% or below

5 Verification and items to confirm

Check	How	Result
INT8 helpers, ranges, calibration limit, per-channel scales, agreement with FP32	28 tests	Pass
Pruning schedule values, 21 and 10 neurons kept, neuron removal keeps the function	Tests	Pass
Stored-number counts, MACs, activation memory	Tests	Pass
TensorFlow script has the specified settings and parses	Static check	Pass
Running build_variants_tf.py, real .tflite files	Needs TensorFlow and tfmot	Not run

- **Baseline.** I used the D1 onset-trained model as M1. Tell me if you want the required D1 model instead; its Critical recall on the 57-window split is 94.1% (16 of 17), just under the Task F3 target.
- **“Filter”.** I took it to mean a hidden neuron, since the network is dense. 35% is applied to each hidden layer; the output layer keeps its 3 neurons.
- **Versions.** TensorFlow 2.14.1 with tfmot 0.8.0 (tested together) and tflite-runtime 2.14.0 match the earlier tasks. The number of neurons tfmot removes may differ from mine by one.

Task F2 — Five-Metric Benchmarking

Latency, p95, file size, accuracy and energy for M1, M2 and M3 | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component F, Module 6

At a glance

Item	Result
Accuracy and Critical recall	**Measured:** 100% for all three variants on the 57-window validation set and on 3,735 independent windows. The test is saturated, so it cannot rank them.
Where they differ	Under input noise, M2 and M3 raise more false alarms than M1: at noise 12σ, 4.2% (M1), 6.6% (M2), 5.4% (M3)
Latency, p95, file size, energy	**Not measured:** they need the TFLite files and your laptop. The harness is ready and checked; Section 5 gives the command.
Size (estimate)	Stored numbers: 3,212 B (M1), 1,568 B (M2), 910 B (M3); the .tflite file adds an estimated 1–3 KB of structure

1 Method

Metric	How it is measured
Mean latency (ms)	200 timed single-window inferences after 10 warm-up runs that are discarded; the harness cycles through validation windows
p95 latency (ms)	95th percentile of the same 200 timings
File size (KB)	Size of the .tflite file (1 KB = 1,000 bytes; bytes are also printed)
Accuracy (%)	The 57-window held-out validation set (stratified 20% split from Task D1), plus Critical recall; also 3,735 independent windows
Energy per inference (mJ)	E = P × t, with P = laptop TDP × CPU% ÷ 100. CPU% is psutil's system-wide reading while inference runs back to back for 2 s; t is the mean latency. Watts × ms = mJ.

I do not have the Lab 2 text, so the energy method above is my reading of "E = P × t with psutil CPU% and laptop TDP estimate". Please confirm.

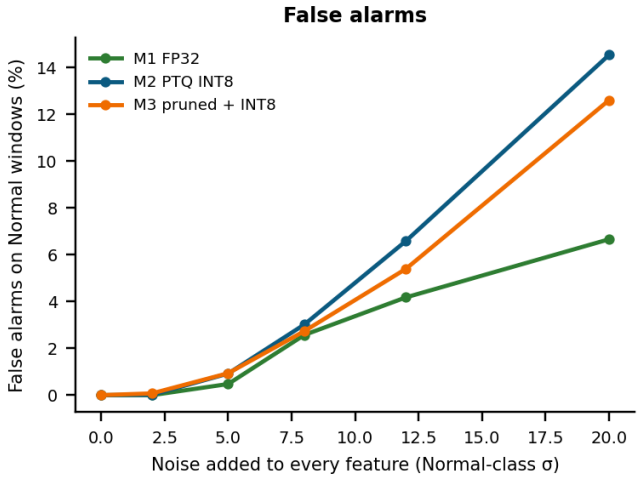
2 Results

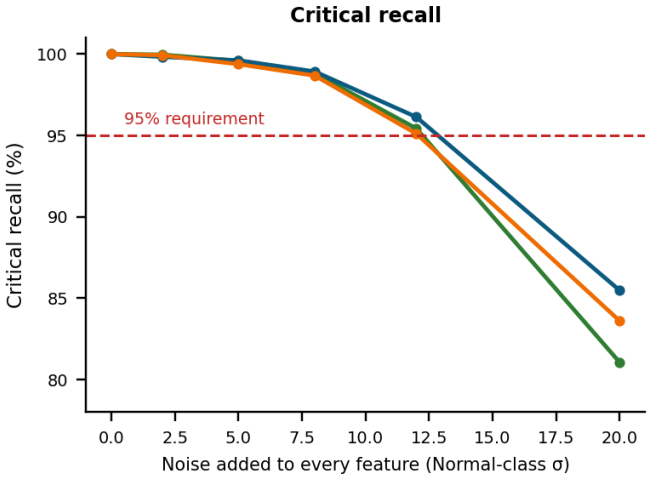
Variant	Mean latency	p95 latency	File size	Accuracy (validation)	Critical recall	Energy
M1 FP32 baseline	measure on laptop	measure on laptop	est. 4.4–5.4 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop
M2 PTQ INT8	measure on laptop	measure on laptop	est. 3.2–4.4 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop
M3 pruned + PTQ INT8	measure on laptop	measure on laptop	est. 2.5–3.7 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop

Amber cells need a TFLite run on your laptop. Accuracy and recall come from numpy code that follows TFLite's integer arithmetic; the real .tflite files should agree within rounding. File sizes are estimates until you run `ls -l`.

Measured on independent runs	M1	M2	M3
Standard runs (1,455 windows): accuracy	100.0%	100.0%	100.0%
Runs where the fault starts mid-run (2,280 windows): accuracy	100.0%	100.0%	100.0%
Critical recall, all 3,735 windows (1,195 Critical)	100.0%	100.0%	100.0%
Agreement with M1's predictions	100%	100.0%	100.0%
Mean time from fault onset to first alert (temperature drift)	20 s	20 s	20 s
Parameters, multiply-accumulates per inference	803, 752	803, 752	400, 366
Peak activation memory	192 B	48 B	31 B

3 Where the variants differ: robustness to noise





Noise (σ)	M1 false alarms	M2	M3	M1 Critical recall	M2	M3
0	0.0%	0.0%	0.0%	100.0%	100.0%	100.0%
5	0.5%	0.9%	0.9%	99.6%	99.6%	99.4%
8	2.6%	3.0%	2.7%	98.8%	98.9%	98.7%
12	4.2%	6.6%	5.4%	95.4%	96.1%	95.1%
20	6.7%	14.5%	12.6%	81.1%	85.5%	83.6%

- Gaussian noise was added to every normalised feature of the 3,735 test windows (5 repeats). One σ is the spread of the Normal class itself.
- INT8 variants raise 1.3 to 1.6 times the false alarms of M1 at 12σ (6.6% and 5.4% against 4.2%) and about 2× at 20σ, because quantisation leaves the Normal range on one or two levels. Critical recall stays similar (M2 slightly higher, M3 slightly lower).
- Critical recall holds above 95% for every variant up to about 12σ of noise and falls to 81–86% at 20σ.

4 Harness self-check (not a result)

Variant (numpy callables)	Mean ms	p95 ms	CPU %	mJ per inference at 15 W	Accuracy
M1	0.0090	0.0153	100	0.1351	100%
M2	0.0446	0.0643	100	0.6685	100%
M3	0.0426	0.0653	100	0.6392	100%

This run only proves the harness works (200 timed runs after 10 warm-ups, p95, psutil CPU%, energy). It used numpy code on the sandbox CPU, where the INT8 emulation is slower than FP32 because it adds rounding steps; TFLite's integer kernels behave differently, so these timings say nothing about TFLite or your laptop. CPU% reads 100 because the sandbox has one core.

5 Running the real benchmark

```
python3 build_variants_tf.py
python3 benchmark.py --tflite M1=m1_fp32.tflite M2=m2_ptq_int8.tflite M3=m3_pruned_int8.tflite --tdp-watts 15
# prints the five metrics per variant and writes benchmark_results.json
```

- Use your laptop's TDP for `--tdp-watts`; 15 W is my placeholder. Close other programs so CPU% reflects the benchmark.
- These models are tiny (752 multiply-accumulates), so I expect all three to be dominated by interpreter overhead and to differ by microseconds. That is an expectation from the operation count, not a measurement.
- Send me benchmark_results.json and I will fill the amber cells and update the Task F3 reasoning.

6 Verification and items to confirm

Check	How	Result
200 timed runs after 10 discarded warm-ups; p95; energy formula; CPU% from psutil	Tests	Pass
Accuracy and Critical recall computed from per-window predictions; int8 input and output handled for .tflite files	Tests, fake interpreter	Pass
Latency, p95, file size and energy on TFLite	Needs TensorFlow and your laptop	Not run

- **Energy method** and **TDP**: confirm both (Section 1).
- **Saturated accuracy.** The simulated modes are very separable, so 100% says little about real trucks; it is the noise test that separates the variants.

Task F3 — Deployment Recommendation

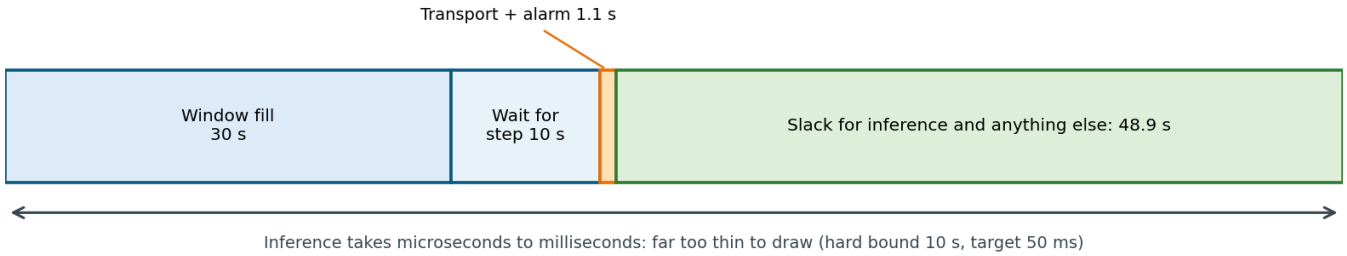
Which model for the 85 trucks? | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component F, Module 6

At a glance

Question	Answer
Which model?	**M1, the FP32 baseline**, retrained with vibration-only runs added (Section 5), on all 85 Raspberry Pi 5 trucks
Why not INT8 or pruned?	On this device and model size they save nothing that matters and cost robustness: M2 and M3 raise 1.3 to 1.6 times the false alarms of M1 under noise (12σ), and add calibration and pruning steps to every update
Latency	Budget 10 s per inference (hard). The operation count suggests microseconds (to be measured), so the 90 s SLA is consumed by the 30 s window and 10 s step, not by inference
Flash and SRAM	No constraint on the Pi 5. Even on the STM32H743 fallback, M1 uses 0.16% of 2 MB flash and 0.02% of 1 MB RAM
Critical recall	All variants exceed 95%: 100% on the validation split (17/17 Critical) and on 1,195 independent Critical windows (95% lower bound 99.7%)
Must fix first	**Every variant calls 100% of vibration-only faults Normal** (bearing wear with normal temperature); the training data had no such case. Adding it fixes this (Section 5).

1 The 90 s SLA as a per-inference latency budget

90 s SLA: from a fault signature appearing in the sensor data to the alert



Item	Time	Source
SLA: fault signature in the data to the alert	90 s	Task A1
Window fill (evidence can take the whole window)	up to 30 s	Task C2
Wait for the next window step	up to 10 s	Task C2
Preprocessing, MQTT and local alarm	about 1.1 s	My allowance; alarm 0.1 s from Task A1
Left for inference	**about 48.9 s**	90 − 41.1
Hard per-inference bound	**10 s**	One inference per 10 s step; slower than that and windows queue up
Engineering target	50 ms	Task A1 allowance

- The binding limit is the 10 s step, not the 48.9 s of slack. Each variant is about 750 multiply-accumulates (366 for M3), so I expect the work to take microseconds (an estimate from the operation count, not yet measured). Latency should not separate the variants.
- Measured in simulation, the first alert came about 20 s after fault onset for all three variants (30 s once for M2), well inside 90 s.
- The TFLite latencies on your laptop and a Pi 5 are still to be measured (Task F2). The decision rule: any variant whose p95 is under 50 ms qualifies.

2 Flash and SRAM for the chosen device

	M1 FP32	M2 INT8	M3 pruned INT8
Stored numbers (the model data in flash)	3,212 B	1,568 B	910 B
Estimated .tflite file	4.4–5.4 KB	3.2–4.4 KB	2.5–3.7 KB
Peak activation memory (SRAM or RAM)	192 B	48 B	31 B

Device	Flash or storage	SRAM or RAM	M1 uses
Raspberry Pi 5, 8 GB + AI HAT+ (chosen in Task B1)	microSD or NVMe, tens of GB (assumed 32 GB)	8 GB LPDDR4X	About 0.00001% of storage and under 0.0001% of RAM
STM32H743 MCU (Task B1 Option 3, fallback)	Up to 2 MB (1 MB on the xG parts)	Up to 1 MB: 192 KB TCM + 864 KB user SRAM	3.2 KB = 0.16% of 2 MB flash; 192 B = 0.02% of RAM

The STM32H743 figures are from ST's datasheet page; the Pi 5 storage size is my assumption. TFLite and interpreter code and your application add to these numbers, and I did not measure them.

If the production model is the 280 KB INT8 model of Task E3: on the Pi 5 it is still trivial, but on the 1 MB-flash MCU 280 KB would be 28% of flash, and an FP32 version (about four times larger, roughly 1.1 MB) would not fit. INT8 would then be needed. That is the one case that flips the decision.

3 Class 2 (Critical) recall

Test	M1	M2	M3	Needs
Validation split (17 Critical windows)	100.0%	100.0%	100.0%	> 95%
Independent runs (1,195 Critical windows)	100.0%	100.0%	100.0%	> 95%
With noise of 8σ added	98.8%	98.9%	98.7%	> 95%
With noise of 12σ added	95.4%	96.1%	95.1%	> 95%

- **M1 meets the target.** 100% on both test sets. The 95% lower bound is 81.6% on the 17-window split (too few windows to prove more) and 99.7% on the independent runs, which is the stronger evidence.
- **The required D1 model would not have passed on the split:** 16 of 17 Critical windows, 94.1%. That is why M1 is the onset-trained model.
- **Recall is by construction Critical = the combined fault.** Vibration-only faults are outside that definition and are missed (Section 5).

4 Decision

Criterion	M1 FP32	M2 PTQ INT8	M3 pruned + INT8
90 s SLA and 10 s bound	Met, by orders of magnitude	Met	Met
Flash and SRAM	Fits	Fits	Fits
Critical recall above 95%	Yes, 100%	Yes, 100%	Yes, 100%
False alarms under noise (12σ)	4.2% (lowest)	6.6%	5.4%
Extra steps on every update	None	Calibrate on current data	Prune, fine-tune, calibrate, validate
Update download (about)	4.4–5.4 KB	3.2–4.4 KB	2.5–3.7 KB
Energy per inference	To be measured	To be measured	To be measured
Verdict	**Deploy**	Only if memory or bandwidth binds	Not worth the extra work

- **Savings are negligible:** M3 downloads about 2 KB less than M1, which across 85 trucks is 0.17 MB or about ₹0.02 per update at ₹0.10/MB.
- **Energy is negligible too, even pessimistically.** If each inference ran the whole 7.5 W Pi for 1 ms (far above what an operation count suggests), a day's 8,640 inferences would use 8,640 × 7.5 mJ = 65 J = 0.018 Wh, which is 0.01% of the 180 Wh the device uses in 24 hours.
- **INT8 buys no accuracy and costs robustness here.** Quantising Normal-class inputs onto one or two levels is the price of the large input range; with no memory or speed problem to solve, it is a risk with no benefit.
- **What would change the decision:** a microcontroller target with a large model (above), measured Pi 5 latency or energy that proves much worse than expected for FP32, or a bandwidth cost that mattered (at ₹0.10/MB it does not).

5 Fix before the 85-truck rollout: vibration-only faults

Test windows (--anomaly vibration)	Flagged Warning or Critical	False alarms on pre-onset windows
D1 models, all of M1, M2, M3 and the required model	0% (87 of 87 predicted Normal)	0%
Retrained with vibration-only runs added (labelled Warning, Task C scheme B)	100.0%	0.0%

- A compressor bearing-wear signal 75σ above normal, with normal temperature, is invisible to the model, because the Task D1 data only had Normal, temperature drift and combined runs. By the Task C table that is an “early refrigeration anomaly” or a unit-failure signature, so it must not be missed.
- Adding the vibration-only runs removes the gap, with no change on the other tests (validation 100.0%, independent runs 100.0%, Critical recall 100.0%). `vibration_gap_check.py` reproduces this.
- Recommended path: add vibration-only runs to the training set, retrain M1, rerun the tests and the E1 reference, then roll out via the shadow-mode process of Task E3.

6 Items to confirm

- **Class for vibration-only faults.** I labelled them Warning (Task C scheme B). If you consider them Critical, tell me.
- **Measured latency and energy.** Run Task F2's benchmark and send benchmark_results.json; the decision rule in Section 1 then closes.
- **Device.** I assumed the Raspberry Pi 5 + AI HAT+ from Task B1. If trucks use the MCU option, the memory section matters and the 280 KB model becomes the question.
- **Production model.** If your real model is the 280 KB INT8 one, FP32 is about four times larger; tell me its size and I will redo the memory and bandwidth numbers.